DarkGenius 26 августа 2013 в 20:22

Поиск подстроки. Алгоритм Кнута–Морриса-Пратта

Алгоритмы*

Из песочницы

В задачах поиска информации одной из важнейших задач является поиск точно заданной подстроки в строке. Примитивный алгоритм поиска подстроки в строке основан на переборе всех подстрок, длина которых равна длине шаблона поиска, и посимвольном сравнении таких подстрок с шаблоном поиска. По традиции шаблон поиска или образец принято обозначать как needle (англ. «иголка»), а строку, в которой ведётся поиск — как haystack (англ. «стог сена»). На языке Python примитивный алгоритм выглядит так:

```
index = -1
for i in xrange(len(haystack)-len(needle)+1):
    success = True
    for j in xrange(len(needle)):
        if needle[j]<>haystack[i+j]:
            success = False
            break
    if success:
        index = i
        break
print index
```

Обозначим n=|haystack|, m=|needle|. Простейший алгоритм поиска даже в лучшем случае

+12 76K 223

O(n+m).

Рассматриваемый далее алгоритм хотя и имеет невысокую скорость на «хороших» данных, но это компенсируется отсутствием регрессии на «плохих». Алгоритм Кнута-Морриса-Пратта является одним из первых алгоритмов с линейной оценкой в худшем случае. Прежде чем перейти к описанию алгоритма, необходимо рассмотреть понятие префикс-функции.

Префикс-функция строки π(S,i) – это длина наибольшего префикса строки S[1..i], который не совпадает с этой строкой и одновременно является ее суффиксом. Проще говоря, это длина наиболее длинного начала строки, являющегося также и ее концом. Для строки S удобно представлять префикс функцию в виде вектора длиной |S|-1. Можно рассматривать префикс-функцию длины |S|, положив π(S,1)=0. Пример префикс функции для строки «abcdabcabcdabcdab»:

S[i]	a	b	c	d	a	b	c	a	b	c	d	a	b	c	d	a	b
π(S,i)	0	0	0	0	1	2	3	1	2	3	4	5	6	7	4	5	6

Предположим, что π(S,i)=k. Отметим следующие свойства префикс-функции.

1. Если S[i+1]=S[k+1], то π(S,i+1)=k+1.
2. S[1..π(S,k)] является суффиксом строки S[1..i]. Действительно, если строка S[1..i] оканчивается строкой S[1… π(S,i)]=S[1..k], а строка S[1..k] оканчивается строкой S[1..π(S,k)], то и строка S[1..i] оканчивается строкой S[1..π(S,k)].
3. ∀ j∈(k,i), S[1..j] не является суффиксом строки S[1..i]. В противном случае было бы неверным предположение π(S,i)=k, так как j>k.

Рассмотренные свойства позволяют получить алгоритм вычисления префикс-функции

Рассмотренные свойства позволяют получить алгоритм вычисления префикс-функции.
Пусть π(S,i)=k. Необходимо вычислить π(S,i+1).

1. Если S[i+1]=S[k+1], то π(S,i+1)=k+1.
2. Иначе, если k=0, то π(S,i+1)=0.
3. Иначе положить k:=π(S,k) и перейти к шагу 1.

Ключевым моментом для понимания сути алгоритма является тот факт, что если найденный на предыдущем шаге суффикс не может быть расширен на следующую позицию, то мы пытаемся рассматривать меньшие суффиксы до тех пор, пока это возможно.

Алгоритм вычисления префикс-функции на языке Python:

```
def prefix(s):
    v = [0]*len(s)
    for i in xrange(1,len(s)):
        k = v[i-1]
        while k > 0 and s[k] <> s[i]:
            k = v[k-1]
        if s[k] == s[i]:
            k = k + 1
        v[i] = k
    return v
```

Покажем, что время работы алгоритма составляет O(n), где n=|S|. Заметим, что асимптотику алгоритма определяет итоговое количество итераций цикла while. Это так, поскольку без учета цикла while каждая итерация цикла for выполняется за время, не превышающее константу. На каждой итерации цикла for k увеличивается не более чем на единицу, значит максимально возможное значение k=n–1. Поскольку внутри цикла while значение k лишь уменьшается, получается, что k не может суммарно уменьшиться больше, чем n–1 раз. Значит цикл while в итоге выполнится не более n раз, что дает итоговую оценку времени алгоритма O(n).

Рассмотрим алгоритм Кнута-Морриса-Пратта, основанный на использовании префикс-функции. Как и в примитивном алгоритме поиска подстроки, образец «перемещается» по строке слева направо с целью обнаружения совпадения. Однако ключевым отличием является то, что при помощи префикс-функции мы можем избежать заведомо бесполезных сдвигов.

Пусть S[0..m–1] – образец, T[0..n–1] – строка, в которой ведется поиск. Рассмотрим сравнение строк на позиции i, то есть образец S[0..m–1] сопоставляется с частью строки T[i..i+m–1]. Предположим, первое несовпадение произошло между символами S[j] и T[i+j], где i < j < m. Обозначим P = S[0..j–1] = T[i..i+j–1]. При сдвиге можно ожидать, что префикс S сойдется с каким-либо суффиксом строки P. Поскольку длина наиболее длинного префикса, являющегося одновременно суффиксом, есть префикс-функция от строки S для индекса j, приходим к следующему алгоритму:

1. Построить префикс-функцию образца S, обозначим ее F.
2. Положить k = 0, i = 0.
3. Сравнить символы S[k] и T[i]. Если символы равны, увеличить k на 1. Если при этом k стало равно длине образца, то вхождение образца S в строку T найдено, индекс вхождения равен i – |S| + 1. Алгоритм завершается. Если символы не равны, используем префикс-функцию для оптимизации сдвигов. Пока k > 0, присвоим k = F[k–1] и перейдем в начало шага 3.
4. Пока i < |T|, увеличиваем i на 1 и переходим в шаг 3.

Возможная реализация алгоритма Кнута-Морриса-Пратта на языке Python выглядит так:

```
def kmp(s,t):
    index = -1
    f = prefix(s)
    k = 0
    for i in xrange(len(t)):
        while k > 0 and s[k] <> t[i]:
            k = f[k-1]
        if s[k] == t[i]:
```

```
        if s[k] == t[i]:
            k = k + 1

        if k == len(s):
            index = i - len(s) + 1
            break
    return index
```

Теги: алгоритмы, поиск подстроки в строке

Хабы: Алгоритмы

Редакторский дайджест
Присылаем лучшие статьи раз в месяц

Электропочта

14 Карма 0 Рейтинг

@DarkGenius

Front-end Engineer at Yandex

Реклама

franchise.edostavka.am

Комментарии 16

ПОХОЖИЕ ПУБЛИКАЦИИ

10 мая в 07:05

Генетический алгоритм поиска решения для задачи по выбору планировок этажа многоквартирного дома

+8 5.6K 50 32 +32

13 апреля в 15:12

Реализуем алгоритм поиска в глубину

+1 9.2K 45 3 +3

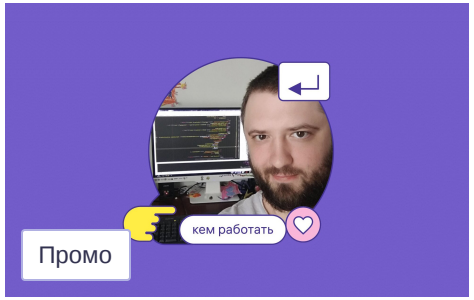
8 ноября 2011 в 18:01

Описание работы алгоритма Shift-OR для поиска подстроки в строке

+42 6.8K 116 16 +16

МИНУТОЧКУ ВНИМАНИЯ

Разместить



Разработчики с Марса, дизайнеры с Венеры

Кем работать в IT в 2022: Frontend-разработчик

Промокод — твой билет в общество потребления

ВАКАНСИИ

IT рекрутер на поиск архитекторов (удаленно)
от 200 000 до 400 000 ₽ · HighTeam · Москва · Можно удаленно

Senior IT Recruiter
от 150 000 ₽ · Benchmark Executive · Можно удаленно

IT рекрутер /copcep
от 50 000 до 150 000 ₽ · АКР · Можно удаленно

Программист 1С
до 200 000 ₽ · ИЦ «Ай-Теко» · Москва

Senior Software Developer, Database Engine (Java or C++)
от 400 000 ₽ · Querify Labs · Можно удаленно

Больше вакансий на Хабр Карьере

ЛУЧШИЕ ПУБЛИКАЦИИ ЗА СУТКИ

вчера в 17:30

Загадочное дело о Raspberry Pi в шкафу для сетевого оборудования

+105 28K 64 38 +38

сегодня в 01:36

JIT-компилятор Python в 300 строк

+77 6.6K 51 14 +14

вчера в 20:15

Запускаем сервис бесплатной аренды отладочных плат с FPGA и не только с FPGA

+65 4.4K 35 21 +21

вчера в 16:50

Термостабилизация телескопа

+60 4.8K 21 26 +26

вчера в 20:36

Как спроектировать фильтр электромагнитных помех для DC-DC преобразователя?

+30 3.3K 60 16 +16

Как устроен МКБ Реактор: история одного стартапа

Мегапост

Реклама

Узнать больше

franchise.edostavka.am

Станьте партнёром СДЭК на особых условиях

Прибыль даже в кризис

ЧИТАЮТ СЕЙЧАС

Я проехал на односкоростном велосипеде 270 км за сутки чтобы показать атлетам, что набор высоты — это условность

5.3K 20 +20

Загадочное дело о Raspberry Pi в шкафу для сетевого оборудования

28K 38 +38

Как мы сравнили «наиболее отечественный» Baikal с Intel

7.6K 41 +41

«Великое увольнение» продолжается: теперь с работы уходят даже боссы

55K 325 +325

Что делать неизвестным в вашем git репозитории

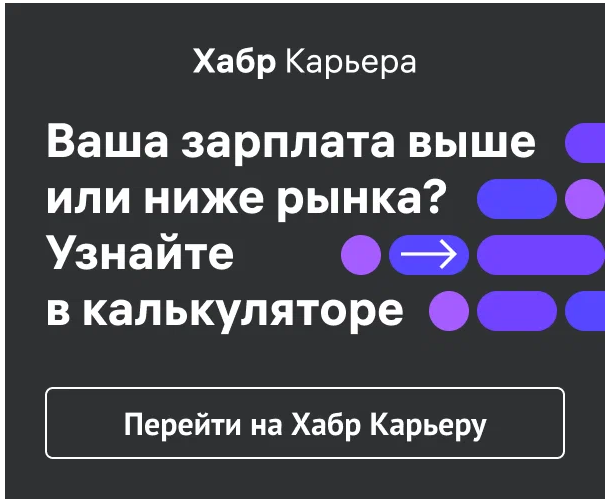
1.8K 4 +4

1.8K 4 +4

Ну-ка повтори: почему разработчики и дизайнеры не понимают друг друга

Мегапост

Реклама



Настройка языка

Техническая поддержка

Вернуться на старую версию

© 2006–2022, Habr